

IF2211 - Strategi Algoritma
Laporan Tugas Kecil 3

Ice Sliding Puzzle Solver



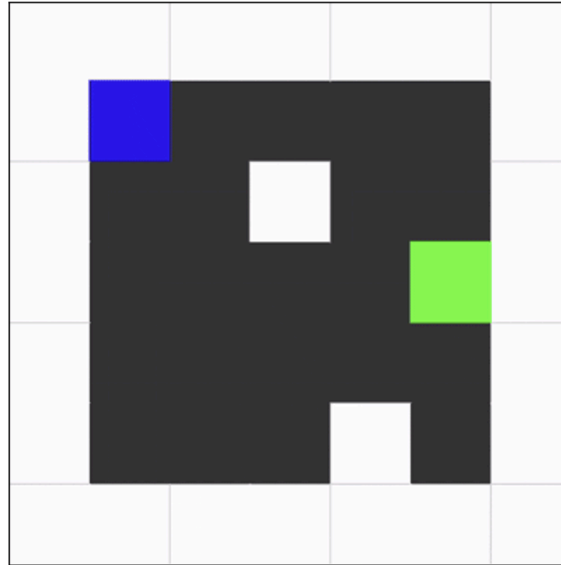
Disusun Oleh:

Keisha Rizka Syofyani	13524073
Helena Kristela Sarhawa	13524109

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

BAB 1

DESKRIPSI TUGAS



Gambar 1 Visualisasi Permainan Ice Sliding Puzzle Solver

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih).

BAB 2

ALGORITMA PATHFINDING

1. Uniform Cost Search (UCS)

Uniform Cost Search (UCS) adalah algoritma pencarian yang menelusuri graf dengan memprioritaskan simpul berdasarkan biaya kumulatif terendah dari titik awal. Algoritma ini merupakan generalisasi dari BFS yang mempertimbangkan bobot pada setiap langkah. Langkah-langkah algoritma UCS yang digunakan pada Ice Sliding Puzzle ini yakni sebagai berikut.

1. Inisialisasi *priority queue* dengan simpul awal. Simpul awal berisi posisi karakter (Z) pada papan dengan nilai *checkpoint* Berikutnya = 0 dan biaya $g = 0$.
2. Selama *priority queue* tidak kosong, ambil simpul dengan biaya g terkecil.
3. Periksa apakah simpul yang diambil sudah pernah dikunjungi dengan biaya yang sama atau lebih kecil. Jika iya, maka lewati simpul ini.
4. Tandai simpul telah dikunjungi dengan biaya g .
5. Periksa apakah kondisi tujuan sudah dipenuhi, yakni posisi aktor berada di tile tujuan (O) dan semua *checkpoint* telah dilewati sesuai urutan. Jika iya, maka rekonstruksi jalur dan kembalikan solusi.
6. Untuk setiap arah yang mungkin dilalui (atas, bawah, kiri, kanan), simulasikan gerakan *sliding* dari posisi saat ini. Gerakan *sliding* dihitung menggunakan *MovementEngine.java* yang menerapkan pengaruh papan es, yakni aktor yang akan terus bergerak hingga menabrak dinding, rintangan, atau keluar papan.
7. Jika gerakan menghasilkan status berhasil, hitung biaya baru yakni biaya g saat ini ditambah akumulasi biaya seluruh tile yang dilalui selama *sliding*.
8. Jika *state* baru belum dikunjungi atau biayanya lebih kecil dari yang pernah dicatat, buat simpul baru dengan biaya g baru dan masukkan ke *priority queue*.
9. Jika *priority queue* habis tanpa menemukan tujuan, maka tidak ada solusi.

2. Greedy Best First Search (GBFS)

Greedy Best First Search (GBFS) adalah algoritma pencarian yang menggunakan fungsi evaluasi berdasarkan estimasi biaya dari simpul saat ini menuju tujuan, tanpa mempertimbangkan biaya yang sudah dikeluarkan dari simpul awal. Algoritma ini bersifat *greedy* karena selalu memilih simpul yang terlihat paling dekat ke tujuan berdasarkan nilai heuristik. Langkah-langkah algoritma GBFS yang digunakan pada Ice Sliding Puzzle ini yakni sebagai berikut.

1. Inisialisasi *priority queue* dengan simpul awal. Simpul awal berisi posisi karakter (Z) pada papan dengan nilai *checkpoint* Berikutnya = 0 dan nilai $f = h(\text{posisi_awal})$ yang dihitung menggunakan heuristik yang dipilih.
2. Selama *priority queue* tidak kosong, ambil simpul dengan nilai $h(n)$ terkecil, yaitu simpul yang diperkirakan paling dekat ke tujuan.
3. Periksa apakah simpul yang diambil sudah pernah dikunjungi. Jika iya, maka lewati simpul ini. Berbeda dengan UCS, GBFS menggunakan set untuk *visited* karena tidak perlu membandingkan biaya.
4. Tandai simpul telah dikunjungi.
5. Periksa apakah kondisi tujuan sudah dipenuhi, yakni posisi aktor berada di tile tujuan (O) dan semua *checkpoint* telah dilewati sesuai urutan. Jika iya, maka

rekonstruksi jalur dan kembalikan solusi.

6. Untuk setiap arah yang mungkin dilalui (atas, bawah, kiri, kanan), simulasikan gerakan *sliding* dari posisi saat ini menggunakan MovementEngine.java.
7. Jika gerakan menghasilkan status berhasil, hitung nilai heuristik $h(\text{state_baru})$ dari *state* hasil gerakan menggunakan heuristik yang dipilih.
8. Jika *state* baru belum pernah dikunjungi, buat simpul baru dengan nilai $f = h(\text{state_baru})$ dan masukkan ke *priority queue*.
9. Jika *priority queue* habis tanpa menemukan tujuan, maka tidak ada solusi.

3. A*

A* adalah algoritma pencarian yang menggabungkan keunggulan UCS dan GBFS dengan menggunakan fungsi evaluasi $f(n) = g(n) + h(n)$ sehingga mempertimbangkan biaya aktual yang sudah dikeluarkan dan estimasi biaya menuju tujuan. Dengan heuristik yang *admissible*, A* menjamin solusi yang optimal. Langkah-langkah algoritma A* yang digunakan pada Ice Sliding Puzzle ini yakni sebagai berikut.

1. Inisialisasi *priority queue* dengan simpul awal. Simpul awal berisi posisi karakter (Z) pada papan dengan nilai checkpointBerikutnya = 0, biaya $g = 0$, dan nilai $f = g + h(\text{posisi_awal})$.
2. Selama *priority queue* tidak kosong, ambil simpul dengan nilai $f(n) = g(n) + h(n)$ terkecil.
3. Periksa apakah simpul yang diambil sudah pernah dikunjungi dengan biaya g yang sama atau lebih kecil. Jika iya, maka lewati simpul ini. A* menggunakan map untuk *visited* agar dapat membandingkan biaya g antar kunjungan ke *state* yang sama.
4. Tandai simpul telah dikunjungi dengan biaya g .
5. Periksa apakah kondisi tujuan sudah dipenuhi, yakni posisi aktor berada di tile tujuan (O) dan semua *checkpoint* telah dilewati sesuai urutan. Jika iya, maka rekonstruksi jalur dan kembalikan solusi.
6. Untuk setiap arah yang mungkin dilalui (atas, bawah, kiri, kanan), simulasikan gerakan *sliding* dari posisi saat ini menggunakan MovementEngine.java.
7. Jika gerakan menghasilkan status berhasil, hitung biaya baru $g(\text{state_baru}) = g(\text{saat ini}) + \text{biaya tile yang dilalui selama sliding}$, lalu hitung nilai $f = g(\text{state_baru}) + h(\text{state_baru})$.
8. Jika *state* baru belum pernah dikunjungi atau biaya g terbaru lebih kecil dari yang pernah dicatat, buat simpul baru dengan nilai f tersebut dan masukkan ke *priority queue*.
9. Jika *priority queue* habis tanpa menemukan tujuan, maka tidak ada solusi.

4. Breadth First Search (BFS) sebagai bonus

Breadth First Search (BFS) adalah algoritma pencarian yang menelusuri graf secara melebar. BFS akan mengeksplorasi semua simpul pada kedalaman tertentu sebelum melanjutkan ke kedalaman berikutnya. BFS menggunakan antrian biasa yang bersifat FIFO sebagai struktur data utamanya. Langkah-langkah algoritma BFS yang digunakan pada Ice Sliding Puzzle ini yakni sebagai berikut.

1. Inisialisasi *queue* (FIFO) dengan simpul awal. Simpul awal berisi posisi karakter (Z) pada papan dengan nilai checkpointBerikutnya = 0 dan biaya $g = 0$.

2. Selama *queue* tidak kosong, ambil simpul paling depan.
3. Periksa apakah simpul yang diambil sudah pernah dikunjungi. Jika iya, maka lewati simpul ini.
4. Tandai simpul telah dikunjungi.
5. Periksa apakah kondisi tujuan sudah dipenuhi, yakni posisi aktor berada di tile tujuan (O) dan semua *checkpoint* telah dilewati sesuai urutan. Jika iya, maka rekonstruksi jalur dan kembalikan solusi.
6. Untuk setiap arah yang mungkin dilalui (atas, bawah, kiri, kanan), simulasikan gerakan *sliding* dari posisi saat ini menggunakan MovementEngine.java.
7. Jika gerakan menghasilkan status berhasil dan *state* baru belum pernah dikunjungi, buat simpul baru dan masukkan ke bagian belakang antrian.
8. Jika *queue* habis tanpa menemukan tujuan, maka tidak ada solusi.

BAB 3

ANALISIS ALGORITMA PATHFINDING

1. Definisi $f(n)$ dan $g(n)$

a. Uniform Cost Search (UCS)

- $f(n) = g(n)$, karena UCS tidak menggunakan heuristik
- $g(n)$ = biaya kumulatif dari *state* awal hingga *state* n , dihitung sebagai jumlah biaya traversal seluruh tile yang dilalui selama gerakan *sliding* sepanjang jalur

b. Greedy Best First Search (GBFS)

- $f(n) = h(n)$, karena GBFS hanya menggunakan estimasi heuristik tanpa mempertimbangkan biaya aktual
- $g(n)$ = biaya kumulatif dari *state* awal hingga *state* n , dihitung tetapi tidak digunakan sebagai prioritas antrian. Nilai g tetap direkam di setiap simpul untuk keperluan pelaporan total biaya solusi yang ditemukan
- $h(n)$ = nilai estimasi heuristik dari *state* n menuju tujuan, dihitung menggunakan heuristik yang dipilih (H1, H2, atau H3)

c. A^*

- $f(n) = g(n) + h(n)$, yaitu estimasi total biaya jalur solusi yang melewati *state* n
- $g(n)$ = biaya kumulatif dari *state* awal hingga *state* n , dihitung sebagai jumlah biaya traversal seluruh tile yang dilalui selama gerakan *sliding* sepanjang jalur
- $h(n)$ = nilai estimasi heuristik dari *state* n menuju tujuan, dihitung menggunakan heuristik yang dipilih (H1, H2, atau H3)

d. Breadth First Search (BFS) sebagai bonus

- $f(n) = g(n)$ karena BFS tidak menggunakan heuristik
- $g(n)$ = biaya kumulatif dari *state* awal hingga *state* n , dihitung dan direkam di setiap simpul untuk keperluan pelaporan total biaya solusi tetapi tidak digunakan sebagai penentu urutan eksplorasi

2. *Admissibility* Heuristik pada A^*

Sebuah heuristik dikatakan *admissible* apabila nilai estimasinya tidak pernah melebihi biaya aktual yang dibutuhkan untuk mencapai tujuan *state* tersebut, atau secara formal $h(n) \leq h^*(n)$ untuk setiap *state* n , dengan $h^*(n)$ adalah biaya aktual optimal.

- H1: menghitung Manhattan distance dari posisi aktor ke checkpoint berikutnya lalu ke tujuan. Pada papan es, aktor harus meluncur minimal sejauh Manhattan distance untuk mencapai suatu titik. Karena setiap tile yang dilalui memiliki biaya minimal 1, dan Manhattan distance menghitung jumlah tile minimum yang harus dilalui, maka H1 tidak pernah melebihi biaya aktual. Maka dari itu, H1 bersifat *admissible*.
- H2: menjumlahkan Manhattan distance di sepanjang rantai checkpoint yang tersisa hingga tujuan. Setiap segmen rantai merupakan *lower bound* dari biaya aktual segmen tersebut sehingga penjumlahan *lower bound* per segmen tetap merupakan *lower bound* total. Maka dari itu, H2 bersifat *admissible*.
- H3: mengalikan nilai H2 dengan biaya tile minimum yang ditemukan di papan. Karena setiap tile memiliki biaya paling sedikit sebesar biaya minimum tersebut, setiap langkah *sliding* akan menghasilkan biaya setidaknya sebesar biaya minimum per tile yang dilewati. H3 akan memberikan *lower bound* yang lebih ketat daripada H2 ketika biaya tile minimum lebih dari 1. Maka dari itu, H3 bersifat *admissible*.

3. Perbandingan Algoritma

Kriteria	UCS	GBFS	A*	BFS (bonus)
Kelengkapan	Lengkap (<i>complete</i>)	Tidak <i>complete</i>	Lengkap (<i>complete</i>)	Lengkap
Optimalitas	Optimal	Tidak optimal	Optimal dengan heuristik admissible	Tidak optimal (hanya optimal jika semua biaya sama)
f(n)	g(n)	h(n)	g(n) + h(n)	g(n)
Eksplorasi	Berdasarkan biaya aktual	Berdasarkan estimasi ke tujuan	Kombinasi UCS dan GBFS	Berdasarkan urutan dimasukannya ke dalam antrian (FIFO)
Kecepatan	Lambat pada ruang state yang besar	Cepat tetapi bisa terjebak	Lebih cepat dari UCS dengan heuristik yang baik	Lambat karena mengeksplorasi semua <i>state</i> per kedalaman
Memori	$O(b^d)$	$O(b^m)$	$O(b^d)$	$O(b^d)$

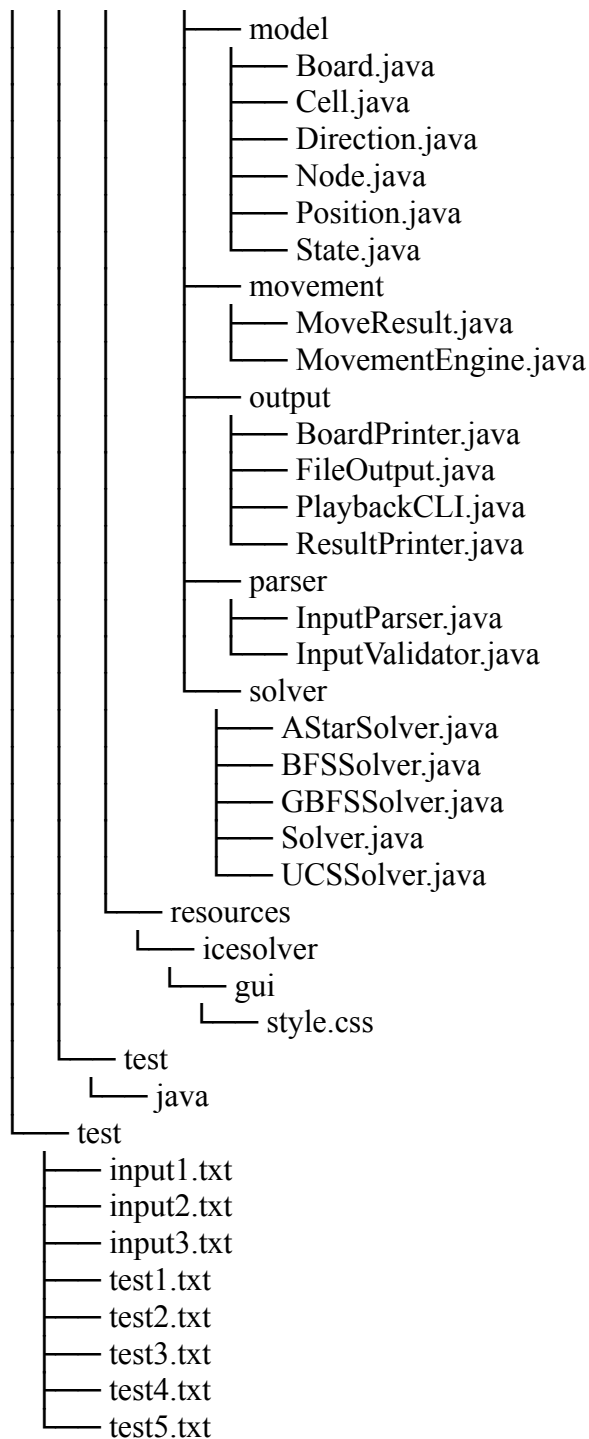
BAB 4

IMPLEMENTASI, PENGUJIAN, DAN ANALISIS HASIL

1. Implementasi

Berikut adalah struktur repositori yang digunakan pada tugas kecil ini.

```
Tucil3_13524073_13524109
├── Makefile
├── README.md
├── Tucil3_13524073_13524109.iml
├── output
│   ├── result1-AStar-H1.txt
│   ├── result1-BFS.txt
│   ├── result1-GBFS-H1.txt
│   ├── result1-UCS.txt
│   ├── result2-AStar-H1.txt
│   ├── result2-BFS.txt
│   ├── result2-GBFS-H1.txt
│   ├── result2-GBFS-H2.txt
│   ├── result2-GBFS-H3.txt
│   ├── result2-UCS.txt
│   ├── result3-AStar-H1.txt
│   ├── result3-BFS.txt
│   ├── result3-GBFS-H1.txt
│   ├── result3-UCS.txt
│   ├── result4-AStar-H1.txt
│   ├── result4-BFS.txt
│   ├── result4-GBFS-H1.txt
│   └── result4-UCS.txt
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── icesolver
│   │   │   │   ├── AppConfig.java
│   │   │   │   ├── Main.class
│   │   │   │   ├── Main.java
│   │   │   ├── gui
│   │   │   │   ├── App.java
│   │   │   │   ├── BoardRenderer.java
│   │   │   │   └── SolverService.java
│   │   └── heuristic
│   │       ├── Heuristic.java
│   │       ├── HeuristicAdvanced.java
│   │       ├── HeuristicCheckpoint.java
│   │       └── HeuristicManhattan.java
```



Program Ice Sliding Puzzle Solver ini diimplementasikan menggunakan bahasa pemrograman Java dengan tampilan GUI menggunakan framework JavaFX. Implementasi program ini terdiri atas beberapa modul utama, yakni:

- Package model
Modul model ini merepresentasikan data utama yang digunakan untuk proses pencarian solusi, yang terdiri atas papan, cell papan, state, node, arah pergerakan,

- dan posisi pemain.
- Package parser
Parser digunakan untuk menangani pembacaan file konfigurasi papan dan menyimpan data papan tersebut sesuai standar representasi data yang digunakan program.
- Package movement
Modul movement ini merepresentasikan mekanisme pergerakan pemain pada papan dengan cara sliding, yang diimplementasikan melalui movement engine.
- Package heuristic
Modul yang mengimplementasikan fungsi heuristic yang digunakan pada algoritma informed search. Heuristic yang diimplementasikan terdiri atas heuristic Manhattan dan dua heuristic tambahan, yakni checkpoint dan advanced.
- Package solver
Modul ini mengimplementasikan algoritma pencarian solusi dari path finding berdasarkan informasi yang tersedia. Adapun algoritma yang digunakan terdiri atas algoritma A Star, GBFS, UCS, dan algoritma dasar BFS.
- Package output
Modul ini bertugas untuk menangani keperluan output dari hasil pencarian solusi, yang terdiri atas pencetakan hasil solusi, pencetakan papan, playback solusi, dan penanganan penyimpanan solusi dalam bentuk file teks.
- Package GUI
GUI dari aplikasi ini diimplementasikan dengan menggunakan JavaFX yang menyediakan tampilan yang lebih intuitif bagi user, yang terdiri atas bagian input dan tampilan hasil pencarian serta visualisasi papan.

Berikut penjelasan beberapa kelas utama dari program ini.

1. Kelas Abstrak Solver

```
package icesolver.solver;
import icesolver.model.*;
import java.util.List;

public abstract class Solver {
    public static final class HasilSolusi {
        public final boolean terpecahkan;
        public final List<Direction> jalur;
        public final List<State> riwayatStatus;
        public final int totalBiaya;
        public final int iterasi;
        public long waktuEksekusiMs;

        public HasilSolusi(boolean terpecahkan,
            List<Direction> jalur, List<State> riwayatStatus, int
            totalBiaya, int iterasi) {
            this.terpecahkan = terpecahkan;
            this.jalur = jalur;
        }
    }
}
```

```

        this.riwayatStatus = riwayatStatus;
        this.totalBiaya = totalBiaya;
        this.iterasi = iterasi;
    }
    public String stringJalur() {
        if (jalur == null || jalur.isEmpty()) return "";
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < jalur.size(); i++) {
            sb.append(jalur.get(i).simbol);
            if(i < jalur.size() - 1){
                sb.append("-");
            }
        }
        return sb.toString();
    }
    public static HasilSolusi tidakAdaSolusi(int iterasi)
{
        return new HasilSolusi(false, null, null, 0,
iterasi);
    }
}
    public abstract HasilSolusi selesaikan(Board papan);
    public abstract String namaAlgoritma();
}

```

Kelas Solver ini digunakan sebagai dasar dari algoritma pencarian solusi lainnya dalam mengimplementasikan program. Kelas ini terdiri atas metode abstrak selesaikan(Board papan) untuk solving path finding pada papan yang diberikan dan namaAlgoritma untuk mengembalikan nama dari algoritma yang sedang bekerja. Kelas ini juga mendefinsikan sebuah kelas HasilSolusi yang digunakan untuk menyimpan hasil pencarian yang didapatkan kemudian, yang terdiri atas keberhasilan pencarian, jalur, riwayat state, total biaya, jumlah iterasi, dan waktu eksekusi yang akan ditampilkan pada bagian output, baik di CLI maupun GUI.

2. Kelas Algoritma UCS

```

package icesolver.solver;
import icesolver.model.*;
import icesolver.movement.*;
import java.util.*;

public class UCSSolver extends Solver {
    @Override

```

```

public HasilSolusi selesaikan(Board papan) {
    PriorityQueue<Node> antrian = new PriorityQueue<>();
    Map<State, Integer> dikunjungi = new HashMap<>();
    State statusAwal = new State(papan.mulai, 0);
    Node nodeAwal = new Node(statusAwal);
    antrian.add(nodeAwal);
    int iterasi = 0;

    long start = System.currentTimeMillis();

    while (!antrian.isEmpty()) {
        Node saat = antrian.poll();
        State status = saat.status;
        if (dikunjungi.containsKey(status) &&
dikunjungi.get(status) <= saat.biayaG) {
            continue;
        }
        dikunjungi.put(status, saat.biayaG);
        iterasi++;
        if (status.isTujuan(papan)) {
            List<Direction> jalur =
saat.rekonstruksiJalur();
            List<State> riwayat =
saat.rekonstruksiStatus();

            long end = System.currentTimeMillis();
            long duration = end - start;
            HasilSolusi result = new HasilSolusi(true,
jalur, riwayat, saat.biayaG, iterasi);
            result.waktuEksekusiMs = duration;
            return result;
        }
        for (Direction arah : Direction.values()) {
            MoveResult hasilGerak =
MovementEngine.geser(papan, status.pos, arah, status);
            if (!hasilGerak.isBerhasil()) continue;
            State statusBaru =
MovementEngine.terapkanGerak(papan, status, arah);
            if (statusBaru == null) continue;
            int biayaBaru = saat.biayaG +

```

```

hasilGerak.biayaGerak;
        if (dikunjungi.containsKey(statusBaru) &&
dikunjungi.get(statusBaru) <= biayaBaru) {
            continue;
        }
        Node anak = new Node(statusBaru, biayaBaru,
biayaBaru, saat, arah);
        antrian.add(anak);
    }
}

    long end = System.currentTimeMillis();
                                HasilSolusi    gagal    =
HasilSolusi.tidakAdaSolusi(iterasi);
    gagal.waktuEksekusiMs = end - start;

    return gagal;
}

@Override
public String namaAlgoritma() {
    return "UCS";
}
}

```

Kelas UCSSolver merupakan kelas yang mengimplementasikan metode penyelesaian pencarian menggunakan algoritma Uniform Cost Search (UCS) yang berdasar atas nilai cost kumulatif. Kelas ini menggunakan PriorityQueue untuk mengurutkan nilai cost kumulatifnya ($g(n)$) untuk menentukan jalur pencarian. Prioritas node yang ditelusuri pada algoritma ini menggunakan fungsi evaluasi $f(n) = g(n)$, dimana $g(n)$ adalah biaya kumulatif node awal menuju node terkini.

3. Kelas Algoritma GBFS

```

package icesolver.solver;
import icesolver.heuristic.*;
import icesolver.model.*;
import icesolver.movement.*;
import java.util.*;

public class GBFSSolver extends Solver {

```

```

private final Heuristic heuristic;

public GBFSSolver(Heuristic heuristic){
    this.heuristic = heuristic;
}

@Override
public HasilSolusi selesaikan(Board papan) {
    PriorityQueue<Node> antrian = new PriorityQueue<>();
    Set<State> dikunjungi = new HashSet<>();
    State statusAwal = new State(papan.mulai, 0);
    Node nodeAwal = new Node(statusAwal, 0,
this.heuristic.perkiraan(statusAwal, papan), null, null);
    antrian.add(nodeAwal);
    int iterasi = 0;

    long start = System.currentTimeMillis();

    while (!antrian.isEmpty()) {
        Node saat = antrian.poll();
        State status = saat.status;
        if (dikunjungi.contains(status)) {
            continue;
        }
        dikunjungi.add(status);
        iterasi++;
        if (status.isTujuan(papan)) {
            List<Direction> jalur =
saat.rekonstruksiJalur();
            List<State> riwayat =
saat.rekonstruksiStatus();

            long end = System.currentTimeMillis();
            long duration = end - start;
            HasilSolusi result = new HasilSolusi(true,
jalur, riwayat, saat.biayaG, iterasi);
            result.waktuEksekusiMs = duration;
            return result;
        }
        for (Direction arah : Direction.values()) {

```

```

                                MoveResult  hasilGerak  =
MovementEngine.geser(papan, status.pos, arah, status);
                                if (!hasilGerak.isberhasil()) continue;
                                State  statusBaru  =
MovementEngine.terapkanGerak(papan, status, arah);
                                if (statusBaru == null) continue;
                                int  biayaBaru  =  saat.biayaG  +
hasilGerak.biayaGerak;
                                if (dikunjungi.contains(statusBaru)) {
                                    continue;
                                }

                                double  biayaF  =
this.heuristic.perkiraan(statusBaru, papan);
                                Node  anak  =  new  Node(statusBaru, biayaBaru,
biayaF, saat, arah);
                                antrian.add(anak);
                            }
                        }

                        long  end  =  System.currentTimeMillis();

                        HasilSolusi  gagal  =
HasilSolusi.tidakAdaSolusi(iterasi);
                        gagal.waktuEksekusiMs  =  end - start;

                        return  gagal;
                    }

    @Override
    public  String  namaAlgoritma() {
        return  "GBFS";
    }
}

```

Kelas GBFSSolver merupakan kelas yang mengimplementasikan metode penyelesaian pencarian menggunakan algoritma Greedy Best First Search (GBFS) yang berdasar atas nilai heuristic untuk memperhitungkan perkiraan cost dari posisi saat ini menuju goal. Nilai heuristic yang digunakan ini didapatkan dari kelas Heuristic, pada tugas ini kami menggunakan 3 jenis heuristic. Pada setiap iterasinya, algoritma akan memilih state yang paling dengan dengan goal. Prioritas node yang ditelusuri pada algoritma ini menggunakan fungsi evaluasi $f(n) = h(n)$, dimana $h(n)$ adalah nilai heuristic perkiraan cost dari node terkini

menuju node goal.

4. Kelas Algoritma A*

```
package icesolver.solver;
import icesolver.heuristic.*;
import icesolver.model.*;
import icesolver.movement.*;
import java.util.*;

public class AStarSolver extends Solver {
    private final Heuristic heuristic;

    public AStarSolver(Heuristic heuristic){
        this.heuristic = heuristic;
    }

    @Override
    public HasilSolusi selesaikan(Board papan) {
        PriorityQueue<Node> antrian = new PriorityQueue<>();
        Map<State, Integer> dikunjungi = new HashMap<>();
        State statusAwal = new State(papan.mulai, 0);
        Node nodeAwal = new Node(statusAwal, 0,
this.heuristic.perkiraan(statusAwal, papan), null, null);
        antrian.add(nodeAwal);
        int iterasi = 0;

        long start = System.currentTimeMillis();

        while (!antrian.isEmpty()) {
            Node saat = antrian.poll();
            State status = saat.status;
            if (dikunjungi.containsKey(status) &&
dikunjungi.get(status) <= saat.biayaG) {
                continue;
            }
            dikunjungi.put(status, saat.biayaG);
            iterasi++;
            if (status.isTujuan(papan)) {
                List<Direction> jalur =
saat.rekonstruksiJalur();
```

```

List<State> riwayat =
saat.rekonstruksiStatus();

        long end = System.currentTimeMillis();
        long duration = end - start;
        HasilSolusi result = new HasilSolusi(true,
jalur, riwayat, saat.biayaG, iterasi);
        result.waktuEksekusiMs = duration;
        return result;
    }
    for (Direction arah : Direction.values()) {
        MoveResult hasilGerak =
MovementEngine.geser(papan, status.pos, arah, status);
        if (!hasilGerak.isberhasil()) continue;
        State statusBaru =
MovementEngine.terapkanGerak(papan, status, arah);
        if (statusBaru == null) continue;
        int biayaBaru = saat.biayaG +
hasilGerak.biayaGerak;
        if (dikunjungi.containsKey(statusBaru) &&
dikunjungi.get(statusBaru) <= biayaBaru) {
            continue;
        }
        double biayaF = biayaBaru +
this.heuristic.perkiraan(statusBaru, papan);
        Node anak = new Node(statusBaru, biayaBaru,
biayaF, saat, arah);
        antrian.add(anak);
    }
}

        long end = System.currentTimeMillis();
        HasilSolusi gagal =
HasilSolusi.tidakAdaSolusi(iterasi);
        gagal.waktuEksekusiMs = end - start;

        return gagal;
    }

@Override

```

```

    public String namaAlgoritma() {
        return "A*";
    }
}

```

Kelas AStarSolver merupakan kelas yang mengimplementasikan metode penyelesaian pencarian menggunakan algoritma A Star, yakni memperhitungkan cost totalnya adalah jumlah dari biaya aktual state asal menuju state saat ini dengan nilai heuristicnya. Algoritma akan memilih kombinasi total nilai cost aktual dan nilai heuristic untuk menentukan state yang akan dipilih selanjutnya. Prioritas node yang ditelusuri pada algoritma ini menggunakan fungsi evaluasi $f(n) = g(n) + h(n)$, dimana $g(n)$ adalah biaya kumulatif node awal menuju node terkini dan $h(n)$ adalah nilai heuristic perkiraan cost dari node terkini menuju node goal. Kombinasi ini dapat menghasilkan solusi optimal dan lebih efisien.

5. Kelas Algoritma BFS

```

package icesolver.solver;
import icesolver.model.*;
import icesolver.movement.*;
import java.util.*;

public class BFSSolver extends Solver {
    @Override
    public HasilSolusi selesaikan(Board papan) {
        Queue<Node> antrian = new LinkedList<>();
        Set<State> dikunjungi = new HashSet<>();
        State statusAwal = new State(papan.mulai, 0);
        Node nodeAwal = new Node(statusAwal);
        antrian.add(nodeAwal);
        int iterasi = 0;

        long start = System.currentTimeMillis();

        while (!antrian.isEmpty()) {
            Node saat = antrian.poll();
            State status = saat.status;
            if (dikunjungi.contains(status)) {
                continue;
            }
            dikunjungi.add(status);
            iterasi++;
            if (status.isTujuan(papan)) {

```

```

                                List<Direction> jalur =
saat.rekonstruksiJalur();

                                List<State> riwayat =
saat.rekonstruksiStatus();

        long end = System.currentTimeMillis();
        long duration = end - start;
        HasilSolusi result = new HasilSolusi(true,
jalur, riwayat, saat.biayaG, iterasi);
        result.waktuEksekusiMs = duration;
        return result;
    }
    for (Direction arah : Direction.values()) {
        MoveResult hasilGerak =
MovementEngine.geser(papan, status.pos, arah, status);
        if (!hasilGerak.isBerhasil()) continue;
        State statusBaru =
MovementEngine.terapkanGerak(papan, status, arah);
        if (statusBaru == null) continue;
        int biayaBaru = saat.biayaG +
hasilGerak.biayaGerak;
        if (dikunjungi.contains(statusBaru)) {
            continue;
        }
        Node anak = new Node(statusBaru, biayaBaru, 0,
saat, arah);
        antrian.add(anak);
    }
}

    long end = System.currentTimeMillis();
                                HasilSolusi gagal =
HasilSolusi.tidakAdaSolusi(iterasi);
    gagal.waktuEksekusiMs = end - start;

    return gagal;
}

@Override
public String namaAlgoritma() {

```

```

        return "BFS";
    }
}

```

Kelas BFSSolver merupakan kelas yang mengimplementasikan metode penyelesaian pencarian menggunakan algoritma dasar pencarian jalur, yakni Breadth First Search (BFS) sebagai algoritma bonus yang kami kerjakan dan digunakan untuk memahami perbandingan algoritma dasar pencarian jalur dengan algoritma khusus pencarian jalur ini dengan pertimbangan cost. Algoritma BFS ini menggunakan konsep eksplorasi FIFO tanpa mempertimbangkan cost untuk menelusuri jalur menuju goal. Algoritma BFS ini hanya mengeksplorasi node berdasarkan kedalaman langkahnya, bukan cumulative traversal cost sehingga solusi yang ditemukan tidak dapat ditentukan memiliki total biaya minimum.

6. Kelas Program Utama CLI

```

package icesolver;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Scanner;

import icesolver.heuristic.Heuristic;
import icesolver.heuristic.HeuristicAdvanced;
import icesolver.heuristic.HeuristicCheckpoint;
import icesolver.heuristic.HeuristicManhattan;
import icesolver.model.Board;
import icesolver.output.BoardPrinter;
import icesolver.output.FileOutput;
import icesolver.output.ResultPrinter;
import icesolver.parser.InputParser;
import icesolver.solver.AStarSolver;
import icesolver.solver.BFSSolver;
import icesolver.solver.GBFSSolver;
import icesolver.solver.Solver;
import icesolver.solver.UCSSolver;

public class Main {
    public static void main(String args[]){
        Scanner input = new Scanner(System.in);
    }
}

```

```

System.out.print(">>> Masukkan file input (.txt) : ");
String fileName = input.nextLine();

Path path;

if(!fileName.endsWith(".txt")){
    path = Paths.get("test/" + fileName + ".txt");
}
else{
    path = Paths.get("test/" + fileName);
}

if(!Files.exists(path)){
    System.out.println("File " + path + " tidak
ditemukan!!");
    return;
}

System.out.println("Pilihan Algoritma yang
Tersedia:");

System.out.println("- UCS");
System.out.println("- GBFS");
System.out.println("- A*");
System.out.println("- BFS");

System.out.print(">>> Masukkan pilihan algoritma : ");
String pilihanAlgoritma = input.nextLine();

// Validation untuk pilihan algoritma
while(!pilihanAlgoritma.equals("UCS") &&
!pilihanAlgoritma.equals("A*") &&
!pilihanAlgoritma.equals("GBFS") &&
!pilihanAlgoritma.equals("BFS")){
    System.out.println("Piliha algoritma tidak
valid.");
    System.out.print(">>> Masukkan pilihan algoritma :
");
    pilihanAlgoritma = input.nextLine();
}

```

```

        String pilihanHeuristic = null;
        if(pilihanAlgoritma.equals("GBFS") ||
pilihanAlgoritma.equals("A*")) {
            System.out.println("Pilihan Heuristic yang
Tersedia :");
            System.out.println("- H1");
            System.out.println("- H2");
            System.out.println("- H3");

            System.out.print(">>> Masukkan pilihan heuristic :
");
            pilihanHeuristic = input.nextLine();
            // Validation untuk pilihan heuristic
            while(!pilihanHeuristic.equals("H1") &&
!pilihanHeuristic.equals("H2") &&
!pilihanHeuristic.equals("H3")) {
                System.out.println("Piliha heuristic tidak
valid.");
                System.out.print(">>> Masukkan pilihan
heuristic : ");
                pilihanHeuristic = input.nextLine();
            }
        }

        System.out.println("\n===== Visualisasi Papan Awal
=====");

        Board papanInput = null;
        BoardPrinter printer = new BoardPrinter();

        try{
            papanInput = InputParser.baca(path.toString());
        } catch(IOException e){
            System.out.println("Papan tidak ditemukan!");
            return;
        }

        if(papanInput != null){
            printer.printBoard(papanInput);

```

```

    }

    System.out.println("Memulai proses pencarian jalur Ice
Sliding dengan algoritma " + pilihanAlgoritma + "...");

    Solver solver = null;
    Heuristic h = null;

    // Pilihan Heuristic
    if(pilihanHeuristic != null){
        if(pilihanHeuristic.equals("H1")) {
            h = new HeuristicManhattan();
        }
        else if(pilihanHeuristic.equals("H2")){
            h = new HeuristicCheckpoint();
        }
        else if(pilihanHeuristic.equals("H3")){
            h = new HeuristicAdvanced();
        }
    }

    // Pilihan Algoritma
    if(pilihanAlgoritma.equals("UCS")){
        solver = new UCSSolver();
    }
    else if(pilihanAlgoritma.equals("GBFS")){
        solver = new GBFSSolver(h);
    }
    else if(pilihanAlgoritma.equals("A*")){
        solver = new AStarSolver(h);
    }
    else if(pilihanAlgoritma.equals("BFS")){
        solver = new BFSSolver();
    }

    Solver.HasilSolusi result =
solver.selesaikan(papanInput);

    if(result == null || !result.terpecahkan){
        System.out.println("Tidak ada solusi yang

```

```

ditemukan.");
        return;
    }

    System.out.println("Solusi yang ditemukan : " +
result.stringJalur());
    System.out.println("Cost dari solusi : " +
result.totalBiaya);
    System.out.println("\n===== Visualisasi Path
Finding =====\n");

    ResultPrinter printResult = new ResultPrinter();
    printResult.printSemuaResult(papanInput, result);

    System.out.println("Waktu eksekusi : " +
String.valueOf(result.waktuEksekusiMs) + " ms");
    System.out.println("Banyak iterasi yang dilakukan : "
+ result.iterasi + " iterasi");

    while(true){
        System.out.print("Apakah Anda ingin melakukan
        playback? (Ya / Tidak) : ");
        String pilihanPlayBack = input.nextLine();

        // Validasi pilihan
        while(!pilihanPlayBack.equals("Ya") &&
!pilihanPlayBack.equals("Tidak")){
            System.out.println("Masukkan tidak valid");
            System.out.print("Apakah Anda ingin melakukan
        playback? (Ya / Tidak) : ");
            pilihanPlayBack = input.nextLine();
        }

        if(pilihanPlayBack.equals("Tidak")){
            break;
        }
        if(pilihanPlayBack.equals("Ya")){
            System.out.print("Pada step berapa Anda ingin
        melakukan playback? (0 - " + result.jalur.size() + ") : ");
            int index = input.nextInt();

```

```

        input.nextLine();
        printResult.printStepResult(papanInput,
result, index);
    }
}

    System.out.print("Apakah Anda ingin menyimpan hasil
pencarian program? (Ya / Tidak) : ");
    String save = input.nextLine();

    while(!save.equals("Ya") && !save.equals("Tidak")){
        System.out.println("Masukkan tidak valid");
        System.out.print("Apakah Anda ingin menyimpan
hasil pencarian program? (Ya / Tidak) : ");
        save = input.nextLine();
    }

    if(save.equals("Ya")){
        System.out.print("Masukkan nama file format txt :
");

        String pathSave = input.nextLine();
        if(!pathSave.endsWith(".txt")){
            pathSave += ".txt";
        }

        pathSave = "output/" + pathSave;
        try{
            FileOutputStream fileOutput = new FileOutputStream();
            fileOutput.simpanHasil(papanInput, solver,
result, pathSave);
        } catch(IOException e) {
            System.out.println("Gagal menyimpan file hasil
pencarian");
            e.printStackTrace();
        }

        System.out.println("File hasil pencarian berhasil
disimpan pada " + pathSave);
    }
    else{

```

```

        System.out.println("Program selesai. Solusi tidak
disimpan.");
    }
}
}

```

Kelas Main bertugas sebagai program utama yang mengelola alur program mulai dari penerimaan input berupa file teks, menerima instruksi user, memanggil fungsi parsing, algoritma pencarian, hingga menampilkan output hasil pencarian pada CLI. Program main ini juga dilengkapi fitur playback untuk menampilkan proses iterasi pada step tertentu. Program utama ini juga dapat menyimpan hasil pencarian ke dalam file teks sehingga history pencarian dapat terdokumentasi dengan baik. Program utama ini juga sudah dilengkapi dengan error handling dan validasi input user untuk menangani kemungkinan kesalahan yang dilakukan user.

7. Kelas Program Utama GUI

```

package icesolver.gui;

import icesolver.model.Board;
import icesolver.output.FileOutput;
import icesolver.parser.InputParser;
import icesolver.solver.Solver;
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.Pagination;
import javafx.scene.control.Separator;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import javafx.stage.FileChooser;

```

```

public class App extends Application {

    @Override
    public void start(Stage stage) {
        // ===== INPUT UI =====

        Label judulInput = new Label("Ice Sliding Puzzle Solver ");
        judulInput.getStyleClass().add("title-label");
        judulInput.setMaxWidth(Double.MAX_VALUE);
        judulInput.setAlignment(Pos.CENTER);

        // INPUT FILE
        Label inputLabel = new Label("Konfigurasi Papan (.txt)");
        inputLabel.getStyleClass().add("section-label");
        Button btnFile = new Button("[Choose file]");
        Label fileNameLabel = new Label("Belum ada file terpilih");
        fileNameLabel.setStyle("-fx-text-fill: #4B5563;" + "-fx-font-size: 14px;");

        // PILIHAN ALGORITMA
        Label algoritmaLabel = new Label("Jenis Algoritma");
        algoritmaLabel.getStyleClass().add("section-label");
        ComboBox<String> pilihanAlgoritma = new
        ComboBox<>(FXCollections.observableArrayList(
            "Uniform Cost Search (UCS)",
            "Greedy Best First Search (GBFS)",
            "A Star (A*)",
            "Breadth First Search (BFS)"));
        pilihanAlgoritma.setValue("Pilih Algoritma");
        pilihanAlgoritma.setStyle("-fx-background-color: #ff79c5;");

        // PILIHAN HEURISTIC untuk GBFS dan A*
        Label heuristicLabel = new Label("Jenis Heuristic");
        heuristicLabel.getStyleClass().add("section-label");
        ComboBox<String> pilihanHeuristic = new
        ComboBox<>(FXCollections.observableArrayList(

```

```

        "H1 : Heuristic Manhattan",
        "H2 : Heuristic Checkpoint",
        "H3 : Heuristic Advanced"));
pilihanHeuristic.setValue("Pilih Heuristic");

// Logika Kondisional Pilihan Heuristic
heuristicLabel.setVisible(false);
pilihanHeuristic.setVisible(false);
pilihanAlgoritma.setOnAction(e -> {
    String selected = pilihanAlgoritma.getValue();
    boolean isHeuristicNeeded = "Greedy Best First
Search (GBFS)".equals(selected) || "A Star
(A*)".equals(selected);
    heuristicLabel.setVisible(isHeuristicNeeded);
    pilihanHeuristic.setVisible(isHeuristicNeeded);
});

// Logika File Chooser
final String[] selectedPath = {null};

btnFile.setOnAction(e -> {
    FileChooser fileInput = new FileChooser();
    fileInput.setTitle("[Choose File]");
    fileInput.getExtensionFilters().add(new
FileChooser.ExtensionFilter("Text Files", "*.txt"));
    var file = fileInput.showOpenDialog(stage);
    if (file != null) {
        fileNameLabel.setText(file.getName());
        selectedPath[0] = file.getAbsolutePath();
    }
});

// Tombol Solver
Button btnSolver = new Button("Run Ice Solver");
btnSolver.getStyleClass().add("main-button");
btnSolver.setAlignment(Pos.CENTER);
HBox tombolBox = new HBox(btnSolver);
tombolBox.setAlignment(Pos.CENTER);

```

```

// Layout
VBox InputUI = new VBox(10); // Jarak antar komponen
adalah 10 pixel
InputUI.setPadding(new Insets(20)); // Margin dari
tepi jendela
InputUI.setAlignment(Pos.TOP_LEFT); // Komponen rata
tengah atas
VBox.setMargin(algoritmaLabel, new Insets(20, 0, 0,
0));
VBox.setMargin(heuristicLabel, new Insets(20, 0, 0,
0));
VBox.setMargin(btnSolver, new Insets(30, 0, 30, 0));
Scene sceneInput = new Scene(InputUI, 800, 600);
InputUI.getChildren().addAll(judulInput, new
Separator(), inputLabel, btnFile, fileNameLabel,
algoritmaLabel, pilihanAlgoritma, heuristicLabel,
pilihanHeuristic, tombolBox);

sceneInput.getStylesheets().add(getClass().getResource("style.
css").toExternalForm());

// ===== Output UI =====
Label judulOutput = new Label("Ice Sliding Puzzle
Solver ");
judulOutput.getStyleClass().add("title-label");
judulOutput.setMaxWidth(Double.MAX_VALUE);
judulOutput.setAlignment(Pos.CENTER);

Label labelResult = new Label("Hasil Pencarian Path");
labelResult.getStyleClass().add("section-label");

Label labelVisualisasi = new Label("Visualisasi
Pencarian Path");
labelVisualisasi.getStyleClass().add("section-label");
labelVisualisasi.setMaxWidth(Double.MAX_VALUE);
labelVisualisasi.setAlignment(Pos.CENTER);

// Footer
Button btnBack = new Button("Back");
btnBack.setAlignment(Pos.CENTER);

```

```

        btnBack.getStyleClass().add("main-button");

        Button btnSave = new Button("Save");
        btnSave.setAlignment(Pos.CENTER);
        btnSave.setDisable(true);
        btnSave.getStyleClass().add("main-button");

        Region spacer = new Region();
        HBox.setHgrow(spacer, Priority.ALWAYS);
        HBox footerBox = new HBox(10);
        footerBox.getChildren().addAll(btnBack, spacer,
btnSave);

        Pagination pagination = new Pagination();
        pagination.setMinHeight(300);
        pagination.setMinWidth(400);

        HBox paginationWrapper = new HBox(pagination);
        paginationWrapper.setAlignment(Pos.CENTER);
        paginationWrapper.setMaxWidth(Double.MAX_VALUE);

        Label labelInfo = new Label();
        labelInfo.setAlignment(Pos.BASELINE_LEFT);
        labelInfo.getStyleClass().add("info-label");

        // Layout
        VBox OutputUI = new VBox(10); // Jarak antar komponen
adalah 10 pixel
        OutputUI.setPadding(new Insets(20)); // Margin dari
tepi jendela
        OutputUI.setStyle("-fx-background-color: #FFF0F6" +
"-fx-text-fill: #ffff;");
        OutputUI.getChildren().addAll(judulOutput, new
Separator(), labelResult, labelInfo, labelVisualisasi,
paginationWrapper, footerBox);
        Scene sceneOutput = new Scene(OutputUI, 800, 600);

sceneOutput.getStylesheets().add(getClass().getResource("style
.css").toExternalForm());

```

```

// Navigasi Back
btnBack.setOnAction(e -> stage.setScene(sceneInput));

// LOGIKA SOLVER
final Board[] lastBoard = {null};
final Solver[] lastSolver = {null};
final Solver.HasilSolusi[] lastResult = {null};

btnSolver.setOnAction(e -> {
    if(selectedPath[0] == null) {
        Alert alert = new
Alert (Alert.AlertType.WARNING);
        alert.setTitle("Warning!");
        alert.setHeaderText(null);
        alert.setContentText("Silakan pilih file
konfigurasi (.txt) terlebih dahulu!");
        alert.showAndWait();
        return;
    }

    if (pilihanAlgoritma.getValue() == null ||
pilihanAlgoritma.getValue().equals("Pilih Algoritma")) {
        Alert alert = new
Alert (Alert.AlertType.WARNING);
        alert.setContentText("Silakan pilih jenis
algoritma terlebih dahulu!");
        alert.showAndWait();
        return;
    }

    boolean butuhHeuristic =
pilihanAlgoritma.getValue().equals("Greedy Best First Search
(GBFS)") || pilihanAlgoritma.getValue().equals("A Star (A*)");

    if (butuhHeuristic && (pilihanHeuristic.getValue()
== null || pilihanHeuristic.getValue().equals("Pilih
Heuristic"))) {
        Alert alert = new
Alert (Alert.AlertType.WARNING);
        alert.setContentText("Silakan pilih jenis

```

```

    heuristic terlebih dahulu!");
        alert.showAndWait();
        return;
    }

    try {
        SolverService service = new SolverService();
        var result = service.solve(selectedPath[0],
        pilihanAlgoritma.getValue(), pilihanHeuristic.getValue());
        Board papanAwal =
        InputParser.baca(selectedPath[0]);

        if(result == null || !result.terpecahkan){
            labelInfo.setText("Solusi tidak
            ditemukan");
        }
        else{
            labelInfo.setText(
                "Path solusi: " +
            result.stringJalur() +
                "\nTotal cost solusi : " +
            result.totalBiaya +
                "\nWaktu eksekusi : " +
            result.waktuEksekusiMs + " ms" +
                "\nBanyak iterasi yang dilakukan
            : " + result.iterasi);

            BoardRenderer renderer = new
            BoardRenderer();

            // Setup Slider

            pagination.setPageCount(result.riwayatStatus.size());

            // Show Papan
            pagination.setPageFactory(pageIndex -> {
                VBox page = new VBox(10);
                page.setAlignment(Pos.CENTER);
                Label stepLabel = new Label();
                stepLabel.setStyle("-fx-font-weight:

```

```

bold;");

        if(pageIndex == 0){
            stepLabel.setText("Initial");
        }
        else{
            stepLabel.setText("Step " +
pageIndex + " : " + result.jalur.get(pageIndex - 1).simbol);
        }

        GridPane papan =
renderer.render(papanAwal,
result.riwayatStatus.get(pageIndex));
        page.getChildren().addAll(stepLabel,
papan);

        return page;
    });
}
lastBoard[0] = papanAwal;
lastSolver[0] = service.getSolver();
lastResult[0] = result;
btnSave.setDisable(false);

stage.setScene(sceneOutput);

} catch (Exception err) {
    err.printStackTrace();
}
});

// Save File
btnSave.setOnAction(e -> {
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Simpan Hasil Solusi");
    fileChooser.getExtensionFilters().add(new
FileChooser.ExtensionFilter("Text Files (*.txt)", "*.txt"));

fileChooser.setInitialFileName("ice-sliding-puzzle-solution.tx
t");

    var file = fileChooser.showSaveDialog(stage);

```

```

        if (file != null) {
            try{
                new FileOutputStream().simpanHasil(lastBoard[0],
lastSolver[0], lastResult[0], file.getAbsolutePath());
            }
            catch(Exception ex){
                ex.printStackTrace();
            }
        }
    });
    stage.setTitle("Ice Solver GUI");
    stage.setScene(sceneInput);
    stage.show();
}

public static void main(String[] args) {
    launch(args);
}
}

```

Kelas App pada program ini berfungsi mengelola program utama yang dapat ditampilkan menggunakan Graphical User Interface (GUI) yang lebih intuitif bagi user saat menjalankan program. GUI ini dikembangkan dengan framework JavaFX yang sudah memiliki banyak library antarmuka yang dapat digunakan untuk mempermudah pengembangan tampilan GUI program. GUI ini menyediakan tampilan mulai dari halaman UI input data yang diberikan user yang sudah disertai penanganan validasi input. GUI ini juga menampilkan halaman output informasi hasil pencarian jalur serta visualisasi papan selama proses pencarian jalur. GUI ini menggunakan styling yang diimplementasikan pada file CSS sebagai resource-nya.

2. Pengujian

Pengujian program dilakukan dengan menggunakan beberapa kasus.

Pengujian Algoritma Pencarian Dasar (Test Case 1)

Algoritma	Input (Konfigurasi Papan)	Output
UCS	5 5 XXXXX	===== HASIL PENCARIAN =====

	XZ**X X***X X**OX XXXXX 0 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0 1 1 1 0 0 0 0 0 0	Algoritma : UCS Hasil pencarian jalur : D-R Total biaya : 4 Jumlah iterasi : 4 Waktu eksekusi : 42 ===== VISUALISASI PENCARIAN ===== Initial XXXXX XZ**X X***X X**OX XXXXX Step 1 : D XXXXX X***X X***X XZ*OX XXXXX Step 2 : R XXXXX X***X X***X X**ZX XXXXX
GBFS, heuristic Manhattan	5 5 XXXXX XZ**X X***X X**OX XXXXX 0 0 0 0 0 0 1 1 1 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : D-R Total biaya : 4 Jumlah iterasi : 3 Waktu eksekusi : 77

	0 1 1 1 0 0 1 1 1 0 0 0 0 0 0	===== VISUALISASI PENCARIAN ===== Initial XXXXXX XZ**X X***X X**OX XXXXXX Step 1 : D XXXXXX X***X X***X XZ*OX XXXXXX Step 2 : R XXXXXX X***X X***X X**ZX XXXXXX
A*, heuristic Manhattan	5 5 XXXXXX XZ**X X***X X**OX XXXXXX 0 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0 1 1 1 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : D-R Total biaya : 4 Jumlah iterasi : 4 Waktu eksekusi : 22 ===== VISUALISASI PENCARIAN ===== Initial XXXXXX XZ**X

		X***X X**OX XXXXX Step 1 : D XXXXX X***X X***X XZ*OX XXXXX Step 2 : R XXXXX X***X X***X X**ZX XXXXX
BFS	5 5 XXXXX XZ**X X***X X**OX XXXXX 0 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0 1 1 1 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : BFS Hasil pencarian jalur : D-R Total biaya : 4 Jumlah iterasi : 4 Waktu eksekusi : 33 ===== VISUALISASI PENCARIAN ===== Initial XXXXX XZ**X X***X X**OX XXXXX Step 1 : D XXXXX

		X***X X***X XZ*OX XXXXX Step 2 : R XXXXX X***X X***X X**ZX XXXXX
--	--	---

Pengujian Algoritma Pencarian Dengan Checkpoint (Test Case 2)

Algoritma	Input (Konfigurasi Papan)	Output
UCS	5 7 XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX 0 0 0 0 0 0 0 1 1 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : UCS Hasil pencarian jalur : R-D Total biaya : 6 Jumlah iterasi : 5 Waktu eksekusi : 35 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX Step 1 : R XXXXXXXX X****ZX

		X*****X X****OX XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X*****X X****ZX XXXXXXXXX
GBFS, heuristic Manhattan	5 7 XXXXXXXXX XZ***0X X*****X X****OX XXXXXXXXX 0 0 0 0 0 0 0 1 1 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-D Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 31 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX XZ***0X X*****X X****OX XXXXXXXXX Step 1 : R XXXXXXXXX X****ZX X*****X X****OX XXXXXXXXX Step 2 : D XXXXXXXXX

		X*****X X*****X X****ZX XXXXXXXX
GBFS, heuristic Checkpoint	5 7 XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX 0 0 0 0 0 0 0 1 1 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-D Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 21 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX Step 1 : R XXXXXXXX X****ZX X*****X X****OX XXXXXXXX Step 2 : D XXXXXXXX X*****X X*****X X****ZX XXXXXXXX
GBFS,	5 7	===== HASIL PENCARIAN

heuristic Advanced	<pre> XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX 0 0 0 0 0 0 0 1 1 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0 </pre>	<pre> ===== Algoritma : GBFS Hasil pencarian jalur : R-D Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 32 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX Step 1 : R XXXXXXXX X****ZX X*****X X****OX XXXXXXXX Step 2 : D XXXXXXXX X*****X X*****X X****ZX XXXXXXXX </pre>
A*, heuristic Manhattan	<pre> 5 7 XXXXXXXX XZ***0X X*****X X****OX XXXXXXXX 0 0 0 0 0 0 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : R-D Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 32 </pre>

	<pre> 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 </pre>	<pre> ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX XZ***0X X*****X X****OX XXXXXXXXX Step 1 : R XXXXXXXXX X****ZX X*****X X****OX XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X*****X X****ZX XXXXXXXXX </pre>
BFS	<pre> 5 7 XXXXXXXXX XZ***0X X*****X X****OX XXXXXXXXX 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : BFS Hasil pencarian jalur : R-D Total biaya : 6 Jumlah iterasi : 5 Waktu eksekusi : 33 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX </pre>

		XZ***0X X*****X X****OX XXXXXXX Step 1 : R XXXXXXX X****ZX X*****X X****OX XXXXXXX Step 2 : D XXXXXXX X*****X X*****X X****ZX XXXXXXX
--	--	--

Pengujian Algoritma Pencarian Multiple Checkpoints (Test Case 3)

Algoritma	Input (Konfigurasi Papan)	Output
UCS	7 9 XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0	===== HASIL PENCARIAN ===== Algoritma : UCS Hasil pencarian jalur : R-D-L-U-R-D Total biaya : 30 Jumlah iterasi : 13 Waktu eksekusi : 76 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXXX XZ*****0X

	0 0 0 0 0 0 0 0	X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 2 : D XXXXXXXXXX X*****X X*****X X*****X X1*****2X X*****ZX XXXXXXXXXX Step 3 : L XXXXXXXXXX X*****X X*****X X*****X X1*****2X XZ*****OX XXXXXXXXXX Step 4 : U XXXXXXXXXX XZ*****X X*****X X*****X X*****2X
--	-----------------	--

		X*****OX XXXXXXXXXX Step 5 : R XXXXXXXXXX X*****ZX X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 6 : D XXXXXXXXXX X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX
GBFS, heuristic Manhattan	7 9 XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-L-D-R-U-D Total biaya : 30 Jumlah iterasi : 9 Waktu eksekusi : 53 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X

		X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 2 : L XXXXXXXXXX XZ*****X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 3 : D XXXXXXXXXX X*****X X*****X X*****X X*****2X XZ*****OX XXXXXXXXXX Step 4 : R XXXXXXXXXX X*****X X*****X X*****X X*****2X X*****ZX XXXXXXXXXX
--	--	---

		Step 5 : U <pre> XXXXXXXXXX X*****ZX X*****X X*****X X*****X X*****OX XXXXXXXXXX </pre> Step 6 : D <pre> XXXXXXXXXX X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX </pre>
GBFS, heuristic Checkpoint	7 9 <pre> XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 000000000 011111110 011111110 011111110 011111110 011111110 000000000 </pre>	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-L-D-R-U-D Total biaya : 30 Jumlah iterasi : 7 Waktu eksekusi : 59 ===== VISUALISASI PENCARIAN ===== Initial <pre> XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX </pre>

		Step 1 : R <pre> XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX </pre> Step 2 : L <pre> XXXXXXXXXX XZ*****X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX </pre> Step 3 : D <pre> XXXXXXXXXX X*****X X*****X X*****X X*****2X XZ*****OX XXXXXXXXXX </pre> Step 4 : R <pre> XXXXXXXXXX X*****X X*****X X*****X X*****2X X*****ZX XXXXXXXXXX </pre> Step 5 : U <pre> XXXXXXXXXX X*****ZX </pre>
--	--	---

		<pre> X*****X X*****X X*****X X*****OX XXXXXXXXXX Step 6 : D XXXXXXXXXX X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX </pre>
GBFS, heuristic Advanced	<pre> 7 9 XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-L-D-R-U-D Total biaya : 30 Jumlah iterasi : 7 Waktu eksekusi : 40 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X </pre>

		X*****X X1*****2X X*****OX XXXXXXXXXX Step 2 : L XXXXXXXXXX XZ*****X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 3 : D XXXXXXXXXX X*****X X*****X X*****X X*****2X XZ*****OX XXXXXXXXXX Step 4 : R XXXXXXXXXX X*****X X*****X X*****X X*****2X X*****ZX XXXXXXXXXX Step 5 : U XXXXXXXXXX X*****ZX X*****X X*****X X*****X X*****OX
--	--	--

		XXXXXXXXXX Step 6 : D XXXXXXXXXX X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX
A*, heuristic Manhattan	7 9 XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : R-D-L-U-R-D Total biaya : 30 Jumlah iterasi : 13 Waktu eksekusi : 63 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX

		Step 2 : D <pre> XXXXXXXXXX X*****X X*****X X*****X X1*****2X X*****ZX XXXXXXXXXX </pre> Step 3 : L <pre> XXXXXXXXXX X*****X X*****X X*****X X1*****2X XZ*****OX XXXXXXXXXX </pre> Step 4 : U <pre> XXXXXXXXXX XZ*****X X*****X X*****X X*****2X X*****OX XXXXXXXXXX </pre> Step 5 : R <pre> XXXXXXXXXX X*****ZX X*****X X*****X X*****2X X*****OX XXXXXXXXXX </pre> Step 6 : D <pre> XXXXXXXXXX </pre>
--	--	--

		X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX
A*, heuristic Checkpoint	7 9 XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : R-D-L-U-R-D Total biaya : 30 Jumlah iterasi : 12 Waktu eksekusi : 23 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 2 : D XXXXXXXXXX X*****X

		X*****X X*****X X1*****2X X*****ZX XXXXXXXXXX Step 3 : L XXXXXXXXXX X*****X X*****X X*****X X1*****2X XZ*****OX XXXXXXXXXX Step 4 : U XXXXXXXXXX XZ*****X X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 5 : R XXXXXXXXXX X*****ZX X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 6 : D XXXXXXXXXX X*****X X*****X X*****X X*****X
--	--	---

		X*****ZX XXXXXXXXXX
A*, heuristic Advanced	7 9 XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : R-D-L-U-R-D Total biaya : 30 Jumlah iterasi : 12 Waktu eksekusi : 43 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 2 : D XXXXXXXXXX X*****X X*****X X*****X X1*****2X X*****ZX

		XXXXXXXXXX Step 3 : L XXXXXXXXXX X*****X X*****X X*****X X1*****2X XZ*****OX XXXXXXXXXX Step 4 : U XXXXXXXXXX XZ*****X X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 5 : R XXXXXXXXXX X*****ZX X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 6 : D XXXXXXXXXX X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX
BFS	7 9	===== HASIL PENCARIAN

	<pre> XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 </pre>	<pre> ===== Algoritma : BFS Hasil pencarian jalur : R-D-L-U-R-D Total biaya : 30 Jumlah iterasi : 13 Waktu eksekusi : 61 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXXX XZ*****0X X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 1 : R XXXXXXXXXX X*****ZX X*****X X*****X X1*****2X X*****OX XXXXXXXXXX Step 2 : D XXXXXXXXXX X*****X X*****X X*****X X1*****2X X*****ZX XXXXXXXXXX Step 3 : L XXXXXXXXXX </pre>
--	---	--

		<pre> X*****X X*****X X*****X X1*****2X XZ*****OX XXXXXXXXXX Step 4 : U XXXXXXXXXX XZ*****X X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 5 : R XXXXXXXXXX X*****ZX X*****X X*****X X*****2X X*****OX XXXXXXXXXX Step 6 : D XXXXXXXXXX X*****X X*****X X*****X X*****X X*****ZX XXXXXXXXXX </pre>
--	--	--

Pengujian Algoritma Pencarian Dengan Pertimbangan Cost (Test Case 4)

Algoritma	Input (Konfigurasi Papan)	Output
-----------	------------------------------	--------

UCS	5 7 XXXXXXXX XZ****X X*****X X****OX XXXXXXXX 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : UCS Hasil pencarian jalur : D-R Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 35 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXX XZ****X X*****X X****OX XXXXXXXX Step 1 : D XXXXXXXX X*****X X*****X XZ***OX XXXXXXXX Step 2 : R XXXXXXXX X*****X X*****X X****ZX XXXXXXXX
GBFS, heuristic Manhattan	5 7 XXXXXXXX XZ****X X*****X X****OX XXXXXXXX 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-D Total biaya : 153 Jumlah iterasi : 3 Waktu eksekusi : 42

	0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	===== VISUALISASI PENCARIAN ===== Initial XXXXXXX XZ****X X*****X X****OX XXXXXXX Step 1 : R XXXXXXX X****ZX X*****X X****OX XXXXXXX Step 2 : D XXXXXXX X*****X X*****X X****ZX XXXXXXX
GBFS, heuristic Checkpoint	5 7 XXXXXXX XZ****X X*****X X****OX XXXXXXX 0 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-D Total biaya : 153 Jumlah iterasi : 3 Waktu eksekusi : 39 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXX XZ****X

		X*****X X****OX XXXXXXX Step 1 : R XXXXXXX X****ZX X*****X X****OX XXXXXXX Step 2 : D XXXXXXX X*****X X*****X X****ZX XXXXXXX
GBFS, heuristic Advanced	5 7 XXXXXXX XZ****X X*****X X****OX XXXXXXX 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : R-D Total biaya : 153 Jumlah iterasi : 3 Waktu eksekusi : 54 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXX XZ****X X*****X X****OX XXXXXXX Step 1 : R XXXXXXX X****ZX

		X*****X X*****OX XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X*****X X*****ZX XXXXXXXXX
A*, heuristic Manhattan	5 7 XXXXXXXXX XZ*****X X*****X X*****OX XXXXXXXXX 0 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : D-R Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 27 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX XZ*****X X*****X X*****OX XXXXXXXXX Step 1 : D XXXXXXXXX X*****X X*****X XZ***OX XXXXXXXXX Step 2 : R XXXXXXXXX X*****X

		X*****X X*****ZX XXXXXXXXX
A*, heuristic Checkpoint	5 7 XXXXXXXXX XZ*****X X*****X X****OX XXXXXXXXX 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : D-R Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 51 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX XZ*****X X*****X X****OX XXXXXXXXX Step 1 : D XXXXXXXXX X*****X X*****X XZ***OX XXXXXXXXX Step 2 : R XXXXXXXXX X*****X X*****X X*****ZX XXXXXXXXX
A*, heuristic Advanced	5 7 XXXXXXXXX XZ*****X X*****X	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : D-R

	X****OX XXXXXXXX 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	Total biaya : 6 Jumlah iterasi : 3 Waktu eksekusi : 40 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXX XZ****X X*****X X****OX XXXXXXXX Step 1 : D XXXXXXXX X*****X X*****X XZ***OX XXXXXXXX Step 2 : R XXXXXXXX X*****X X*****X X****ZX XXXXXXXX
BFS	5 7 XXXXXXXX XZ****X X*****X X****OX XXXXXXXX 0 0 0 0 0 0 0 1 50 50 50 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0	===== HASIL PENCARIAN ===== Algoritma : BFS Hasil pencarian jalur : D-R Total biaya : 6 Jumlah iterasi : 4 Waktu eksekusi : 36 ===== VISUALISASI PENCARIAN =====

		Initial XXXXXXXX XZ****X X*****X X****OX XXXXXXXX Step 1 : D XXXXXXXX X*****X X*****X XZ***OX XXXXXXXX Step 2 : R XXXXXXXX X*****X X*****X X****ZX XXXXXXXX
--	--	--

Pengujian Algoritma Pencarian Kompleks (Test Case 5)

Algoritma	Input (Konfigurasi Papan)	Output
UCS	7 7 XXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999	===== HASIL PENCARIAN ===== Algoritma : UCS Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 13 Waktu eksekusi : 51 ===== VISUALISASI PENCARIAN ===== Initial

999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999	XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX Step 1 : U XXXXXXXXX XZ****X X**X**X X****OX X1***LX X***X*X XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXXXX Step 3 : R XXXXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXXXX Step 4 : U XXXXXXXXX X*****X X**X**X
--	---

		X**Z*OX X****LX X***X*X XXXXXXX Step 5 : R XXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXX
GBFS, heuristic Manhattan	7 7 XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999	===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 6 Waktu eksekusi : 37 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX Step 1 : U XXXXXXX XZ****X X**X**X X****OX

		X1***LX X***X*X XXXXXXX Step 2 : D XXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXX Step 3 : R XXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXX Step 4 : U XXXXXXX X*****X X**X**X X**Z*OX X****LX X***X*X XXXXXXX Step 5 : R XXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXX
--	--	---

GBFS, heuristic Checkpoint	<pre> 7 7 XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 6 Waktu eksekusi : 36 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX Step 1 : U XXXXXXXXX XZ****X X**X**X X****OX X1***LX X***X*X XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXXXX Step 3 : R </pre>
----------------------------------	--	--

		<pre> XXXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXXX Step 4 : U XXXXXXXX X*****X X**X**X X**Z*OX X****LX X***X*X XXXXXXXX Step 5 : R XXXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXXX </pre>
GBFS, heuristic Advanced	<pre> 7 7 XXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : GBFS Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 6 Waktu eksekusi : 37 ===== VISUALISASI PENCARIAN ===== Initial </pre>

999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999	XXXXXXXXX X0***X X**X**X X***OX X1***LX XZ**X*X XXXXXXXXX Step 1 : U XXXXXXXXX XZ***X X**X**X X***OX X1***LX X***X*X XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X**X**X X***OX X***LX XZ**X*X XXXXXXXXX Step 3 : R XXXXXXXXX X*****X X**X**X X***OX X***LX X**ZX*X XXXXXXXXX Step 4 : U XXXXXXXXX X*****X X**X**X
--	---

		X**Z*OX X****LX X***X*X XXXXXXX Step 5 : R XXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXX
A*, heuristic Manhattan	7 7 XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999	===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 13 Waktu eksekusi : 30 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX Step 1 : U XXXXXXX XZ****X X**X**X X****OX

		X1***LX X***X*X XXXXXXX Step 2 : D XXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXX Step 3 : R XXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXX Step 4 : U XXXXXXX X*****X X**X**X X**Z*OX X****LX X***X*X XXXXXXX Step 5 : R XXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXX
--	--	---

A*, heuristic Checkpoint	<pre> 7 7 XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 13 Waktu eksekusi : 33 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX Step 1 : U XXXXXXXXX XZ****X X**X**X X****OX X1***LX X***X*X XXXXXXXXX Step 2 : D XXXXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXXXX Step 3 : R </pre>
--------------------------------	--	--

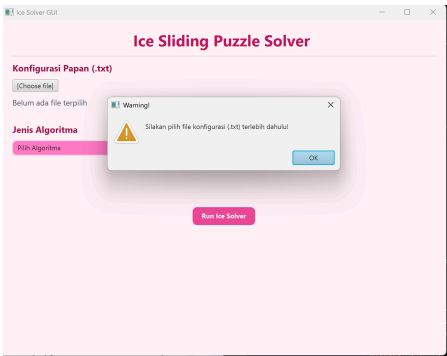
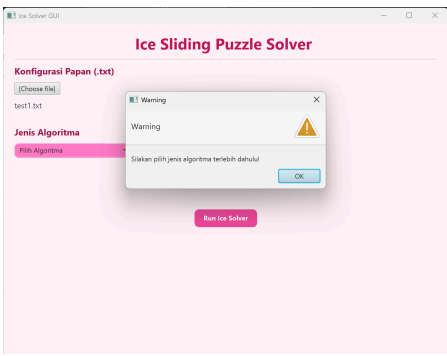
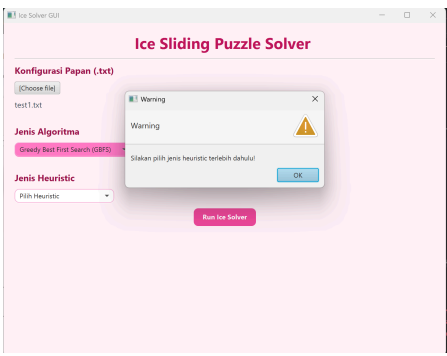
		<pre> XXXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXXX Step 4 : U XXXXXXXX X*****X X**X**X X**Z*OX X****LX X***X*X XXXXXXXX Step 5 : R XXXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXXX </pre>
A*, heuristic Advanced	<pre> 7 7 XXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 </pre>	<pre> ===== HASIL PENCARIAN ===== Algoritma : A* Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 13 Waktu eksekusi : 30 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXX </pre>

	999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999	X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX Step 1 : U XXXXXXX XZ****X X**X**X X****OX X1***LX X***X*X XXXXXXX Step 2 : D XXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXX Step 3 : R XXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXX Step 4 : U XXXXXXX X*****X X**X**X X**Z*OX
--	--	--

		X****LX X***X*X XXXXXXXXX Step 5 : R XXXXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXXXX
BFS	7 7 XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999	===== HASIL PENCARIAN ===== Algoritma : BFS Hasil pencarian jalur : U-D-R-U-R Total biaya : 68 Jumlah iterasi : 15 Waktu eksekusi : 33 ===== VISUALISASI PENCARIAN ===== Initial XXXXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXXXX Step 1 : U XXXXXXXXX XZ****X X**X**X X****OX X1***LX

		X***X*X XXXXXXXX Step 2 : D XXXXXXXX X*****X X**X**X X****OX X****LX XZ**X*X XXXXXXXX Step 3 : R XXXXXXXX X*****X X**X**X X****OX X****LX X**ZX*X XXXXXXXX Step 4 : U XXXXXXXX X*****X X**X**X X**Z*OX X****LX X***X*X XXXXXXXX Step 5 : R XXXXXXXX X*****X X**X**X X****ZX X****LX X***X*X XXXXXXXX
--	--	--

Pengujian Validasi Program

Validasi	GUI	CLI
Nama file / file input		>>> Masukkan file input (.txt) : tt File test/tt.txt tidak ditemukan!!
Pilihan algoritma		>>> Masukkan file input (.txt) : test1 Pilihan Algoritma yang Tersedia: - UCS - GBFS - A* - BFS >>> Masukkan pilihan algoritma : kk Pilih algoritma tidak valid. >>> Masukkan pilihan algoritma : UCS
Pilihan Heuristic		>>> Masukkan file input (.txt) : test1 Pilihan Algoritma yang Tersedia: - UCS - GBFS - A* - BFS >>> Masukkan pilihan algoritma : GBFS Pilihan Heuristic yang Tersedia : :

		- H1 - H2 - H3 >>> Masukkan pilihan heuristic : hh Piliha heuristic tidak valid. >>> Masukkan pilihan heuristic :
Pilihan playback		Apakah Anda ingin melakukan playback? (Ya / Tidak) : kk Masukkan tidak valid Apakah Anda ingin melakukan playback? (Ya / Tidak) : Ya Pada step berapa Anda ingin melakukan playback? (0 - 2) : 1 Step 1 : D XXXXX X***X X***X XZ*OX XXXXX Apakah Anda ingin melakukan playback? (Ya / Tidak) :
Pilihan save file		Apakah Anda ingin menyimpan hasil pencarian program? (Ya / Tidak) : kkk Masukkan tidak valid Apakah Anda ingin menyimpan hasil pencarian program? (Ya / Tidak) :

3. Analisis Hasil

Pada kasus uji 1, dilakukan pengujian algoritma terhadap kasus pencarian sederhana untuk menemukan path ke goal tanpa adanya checkpoint, tembok, maupun lava. Pada kasus ini, seluruh algoritma berhasil menemukan goal dengan jalur yang sama, yakni D-R dengan total biaya 4. Hal ini terjadi karena faktor peta yang hanya memiliki 1 jalur optimal dan tidak adanya checkpoint yang harus dikunjungi serta cost seluruh tile sama. Adapun tujuan dari pengujian ini adalah untuk memastikan seluruh algoritma pencarian dapat berfungsi dengan

benar tanpa bug. Pada perbandingannya, terlihat bahwa algoritma BFS, UCS, dan A* melakukan iterasi sebanyak 4 kali untuk menemukan jalur optimal, namun algoritma GBFS hanya melakukan iterasi sebanyak 3 kali. Hal ini membuktikan bahwa algoritma GBFS lebih efisien dibandingkan algoritma lain. Hal ini dikarenakan GBFS dipengaruhi oleh kombinasi 2 faktor untuk menentukan pertimbangan node yang akan dikunjungi, yakni nilai aktual cost dan juga nilai heuristic.

Pada kasus uji 2, pengujian dilakukan dengan menggunakan kasus uji dengan adanya sebuah checkpoint yang harus dikunjungi saat pencarian jalur ke goal. Seluruh algoritma menghasilkan solusi jalur yang sama dan dengan cost total yang sama, yakni 6 dengan memastikan jalur yang ditemukan melewati checkpoint. Namun iterasi yang dilakukan algoritma UCS dan BFS sebanyak 5, sedangkan iterasi pada algoritma A* dan GBFS hanya 3 iterasi. Jalur yang dihasilkan dari ketiga heuristic pada algoritma GBFS sama. Hal ini menunjukkan bahwa heuristic yang digunakan berfungsi dengan benar dan tidak menimbulkan ambiguitas.

Pada kasus uji 3, pengujian dilakukan dengan meningkatkan kompleksitas pada kasus uji 2, yakni dengan menambahkan beberapa checkpoint yang harus dikunjungi untuk memastikan urutan pengunjungan checkpoint sudah dilakukan dengan benar. Hasil pengujian menunjukkan bahwa semua algoritma berhasil menemukan jalur menuju goal dengan total biaya 30, namun melalui jalur yang bervariasi. Algoritma UCS, BFS, dan A* menghasilkan jalur R-D-L-U-R-D, sedangkan algoritma GBFS menghasilkan jalur R-L-D-R-U-D. Dari hasil pencarian, terlihat bahwa iterasi yang dihasilkan pada algoritma GBFS paling sedikit dibandingkan algoritma yang lain dan penggunaan heuristic Checkpoint dan Advanced menghasilkan iterasi paling minimum. Hal ini dikarenakan heuristic manhattan hanya mempertimbangkan jarak ke goal, sedangkan heuristic checkpoint dan advanced juga mempertimbangkan checkpoint yang belum dikunjungi sehingga pencarian lebih terarah.

Pada kasus uji 4, pengujian dilakukan dengan tujuan melihat bagaimana algoritma bekerja untuk menemukan solusi dengan cost yang paling efisien. Pada konfigurasi papan, terdapat tile dengan cost yang tinggi, yakni 50 sehingga jalur terpendek yang diambil belum tentu merupakan jalur termurah. Dari hasil pencarian, terlihat bahwa semua algoritma berhasil menghasilkan solusi jalur yang berbeda-beda. Algoritma UCS, A*, dan BFS menelusuri dengan jalur D-R dengan cost 6, sedangkan GBFS menelusuri dengan jalur R-D dengan cost paling tinggi, yakni 153. Hal ini sesuai karakteristik UCS yang selalu memilih node dengan akumulasi cost terkecil. Pada kasus ini, UCS dan A* berhasil menemukan jalur terdekat dengan cost paling murah dan iterasi paling sedikit. Walaupun terdapat jalur lain yang secara posisi terlihat lebih dekat ke goal, UCS dan A* mampu menghindari tile berbiaya tinggi dan memilih jalur yang lebih ekonomis. Jumlah iterasi yang dibutuhkan hanya 3 iterasi, menunjukkan bahwa pencarian cukup efisien pada kasus ini.

Pada kasus uji terakhir, konfigurasi papan dibuat lebih kompleks, dengan adanya beberapa checkpoint yang harus dilalui, tembok, dan lava serta dengan cost yang bervariasi. Pada kasus ini, semua algoritma berhasil menemukan jalur yang sama, yakni U-D-R-U-R dengan total cost 68. Namun pada kasus ini GBFS berhasil melakukan iterasi paling sedikit, yakni 6 kali dibandingkan algoritma lain yang melakukan 13 iterasi. Hal ini menunjukkan GBFS

memberikan performa terbaik dengan iterasi paling sedikit. Sedangkan BFS merupakan algoritma yang paling kurang efisien karena melakukan iterasi paling banyak, yakni sebanyak 15.

BAB 5

KESIMPULAN DAN SARAN

1. Kesimpulan

Program Ice Sliding Solver yang melakukan pencarian jalur menuju goal yang diberikan pada tugas kecil 3 Strategi Algoritma ini berhasil diimplementasikan dan dijalankan dengan baik. Program ini diimplementasikan menggunakan bahasa pemrograman Java dengan antarmuka GUI menggunakan framework JavaFX. Program yang diimplementasikan dapat membaca file konfigurasi papan, melakukan validasi input, menjalankan proses pencarian jalur menggunakan algoritma UCS, GBFS, A*, dan BFS. Program ini juga dapat menampilkan visualisasi solusi jalur yang ditemukan, menampilkan ringkasan informasi hasil pencarian, serta menyimpan hasil solusi sebagai dokumentasi hasil pencarian dalam file teks. Berdasarkan hasil pengujian yang dilakukan terhadap berbagai jenis kasus uji, program ini berhasil menyelesaikan puzzle jalur menggunakan algoritma dan menghasilkan jalur yang valid tanpa error.

Setiap algoritma memiliki karakteristik dan performa yang berbeda-beda. UCS secara konsisten menghasilkan solusi optimal pada kasus dengan biaya tile yang bervariasi, namun membutuhkan eksplorasi state yang relatif lebih besar. GBFS mampu memberikan waktu pencarian dan jumlah iterasi yang lebih kecil, tetapi tidak selalu menghasilkan solusi optimal. A* memberikan keseimbangan terbaik antara optimalitas dan efisiensi karena menggabungkan biaya aktual dan heuristic. Sementara itu, BFS dapat menemukan solusi dengan baik pada kasus sederhana, namun kurang efisien pada ruang state yang lebih kompleks. Dengan adanya heuristic yang berbeda-beda juga mempengaruhi performa pencarian jalur yang dilakukan.

2. Saran

Berdasarkan pengerjaan tugas kecil ini, terdapat beberapa saran yang diperoleh. Pertama, heuristic yang lebih adaptif dapat dikembangkan terhadap pola papan es, misalnya ikut mempertimbangkan posisi dinding dan rintangan. Kedua, *testing* pada papan yang lebih besar dan kompleks dapat dilakukan untuk menganalisis performa setiap algoritma. Terakhir, algoritma lain dapat ditambahkan untuk membandingkan algoritma mana yang memberikan performa paling baik.

LAMPIRAN

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

Tautan Repository GitHub: https://github.com/keirzka/Tucil3_13524073_13524109.git

DAFTAR PUSTAKA

- Munir, R. 2026. “Penentuan rute (Route/Path Planning) - Bagian 1”. [Online]. Tersedia: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-(2026)-Bagian1.pdf). [2 Mei 2026].
- Munir, R. 2026. “Penentuan rute (Route/Path Planning) - Bagian 2”. [Online]. Tersedia: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-(2026)-Bagian2.pdf). [2 Mei 2026].

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Keisha Rizka Syofyani



Helena Kristela Sarhawa